ClearURL: Explainable AI-Based Phishing URL Detection System

Technical Documentation

FIT5238 Group 15

August, 2026

Iteration 1 Submission Record

Contents

1 Introduction **2**
- 1.1 Deployed Access URL . . . 2
- 1.2 Scope of Iteration 1 . . . 2

2 System Architecture **2**
- 2.1 Architectural Rationale . . . 3

3 Functional Design **3**
- 3.1 Authentication and Access Control . . . 3
- 3.2 Employee Analysis Workflow . . . 3
- 3.3 Suspicious Website Reporting . . . 4
- 3.4 Admin Monitoring Workflow . . . 4

4 AI Model Integration **4**
- 4.1 Model Selection . . . 4
- 4.2 Training Data and Preparation . . . 4
- 4.3 Feature Set . . . 5
- 4.4 Prediction Logic . . . 5
- 4.5 Model Thresholds . . . 6
- 4.6 Recorded Test Metrics . . . 6

5 Backend Design **6**
- 5.1 Application Entry Point . . . 6
- 5.2 API Endpoints . . . 6
- 5.3 Error Handling . . . 7

6 Persistence Design **7**
- 6.1 Database Layout . . . 7
- 6.2 Stored Data Semantics . . . 7

7 Frontend Design **8**
- 7.1 User Experience Strategy . . . 8
- 7.2 Score Presentation Logic . . . 8

8 Testing and Validation **8**
- 8.1 Automated Validation . . . 8
- 8.2 Recorded Evidence . . . 8

9 Deployment and Runtime **9**
- 9.1 Local Runtime . . . 9
- 9.2 Operational Notes . . . 9

10 Limitations and Future Improvements **9**

11 Conclusion **9**

1 Introduction

ClearURL is a full-stack phishing URL detection system designed for SME users. The system allows an employee to submit a suspicious URL, receive a safety-oriented assessment, review machine-generated explanations, and report the site for administrative review. An authenticated admin user can review recent scans, inspect suspicious cases, and inspect model evaluation records. The implementation is intentionally modular at the code level, but the delivered prototype is exposed to users as one integrated product with a single frontend and a single backend service.

1.1 Deployed Access URL

The current deployed prototype is publicly accessible at:

`http://120.26.118.110/`

This URL is the preferred access method for project demonstration and assessment because it allows the system to be used directly without requiring the assessor to install local dependencies or run backend and frontend services manually.

1.2 Scope of Iteration 1

The first working release includes the following capabilities:

- authentication for employee and admin users
- URL submission and preprocessing
- lexical and webpage feature extraction
- phishing prediction using a trained machine learning model
- explanation generation and recommendation output
- suspicious website reporting and case aggregation
- admin monitoring, scan history, and model visibility

This scope matches the first usable prototype required for the project: a user can log in, analyse a URL, receive a result, and have the result recorded for later admin review. From a software engineering perspective, this release is the minimum integrated product increment that demonstrates both user value and a persistent end-to-end system state.

2 System Architecture

ClearURL follows a layered client-server design:

- **Presentation layer**: React + Vite + TypeScript frontend
- **Application layer**: Starlette backend APIs
- **AI pipeline layer**: preprocessing, feature extraction, prediction, explainability
- **Persistence layer**: multiple SQLite databases
- **Model artifact**: serialized XGBoost bundle stored under `artifacts/`

The frontend communicates with the backend through JSON REST endpoints. The backend is the single integration point that orchestrates preprocessing, feature extraction, model inference, explanation creation, logging, and feedback persistence. This structure keeps the employee workflow simple while still allowing the internal services to be tested independently.

The main runtime entry point is `app.main:app`, which registers the authentication, analysis, reporting, and admin routes. During local development the frontend runs on Vite and forwards API calls to the backend, while production deployment can place both layers behind a reverse proxy.

2.1 Architectural Rationale

The current implementation intentionally uses one integrated backend service rather than multiple separately deployed services. This reduces operational complexity during the first iteration and makes the machine learning pipeline easier to validate under real user interaction. At the same time, the internal code remains separated by responsibility, which keeps the codebase maintainable and supports later refactoring if a more distributed deployment is required.

3 Functional Design

3.1 Authentication and Access Control

Authentication is implemented with a role-based session model. The system seeds two local accounts:

- `employee@clearurl.local` for employee access
- `admin@clearurl.local` for administrative access

Passwords are stored as PBKDF2-SHA256 hashes in the authentication database. Successful login returns a signed session token that is required for protected API calls. Employee users are restricted to employee functions, while admin users are required for admin endpoints. This design prevents unauthorized access to the monitoring dashboard and makes the employee and admin experiences clearly separated. The session repository also manages session expiry, which reduces the risk of stale authenticated sessions persisting indefinitely in a browser.

3.2 Employee Analysis Workflow

The employee workflow is implemented as a single end-to-end analysis pipeline:

1. The employee enters a URL in the frontend.
2. The frontend performs a basic browser-level format check.
3. The frontend sends the URL to `POST /api/analyze`.
4. The backend validates the session and parses the request body.
5. The URL is normalized and validated.
6. Web content is fetched when possible to collect HTML and redirect-chain evidence.
7. URL and webpage features are extracted into a fixed feature schema.
8. The trained model generates a phishing probability.
9. Risk guardrails adjust the final internal risk score when evidence is incomplete or strongly benign.
10. The explainability layer converts technical feature impacts into employee-friendly reasons.
11. The prediction is logged in the scan history database.
12. The frontend displays the result as a safety score, risk category, reasons, and recommendations.

The implementation deliberately returns both the preprocessing details and the final prediction payload so that later debugging and testing can inspect each stage of the analysis chain. This also makes it easier to compare the frontend view with backend logs during validation.

The analysis result shown to the employee is safety-oriented: higher values indicate a safer URL. At the backend level, the internal logging still stores the model-oriented risk score together with the final classification.

3.3 Suspicious Website Reporting

When the employee clicks *Report this website*, the frontend submits the analysis request identifier, normalized URL, and optional comment to `POST /api/reports`. The reporting service verifies that the report matches a stored analysis record before saving it. Repeated reports for the same URL are aggregated into a single reviewable case so that the admin view remains operationally useful. This aggregation avoids clutter in the admin console and lets the team focus on unique suspicious targets rather than duplicate submissions. It also creates more meaningful monitoring data because report frequency is preserved without duplicating the case list.

3.4 Admin Monitoring Workflow

The admin console is populated through backend APIs that read the scan history, feedback store, and model records. The admin dashboard shows:

- total scan counts and risk distribution
- recent scan history
- aggregated suspicious cases
- model version listings
- evaluation metrics and benchmark records

The admin view is read-only in the current release and is intended for monitoring rather than manual case resolution. The current implementation is enough to support demo-time inspection of recent activity, suspicious case trends, and model quality without requiring a separate admin backend. This is appropriate for the first iteration because it delivers operational visibility without introducing a more complex review workflow prematurely.

4 AI Model Integration

4.1 Model Selection

The production artifact currently deployed in the repository is an XGBoost model stored at:

`artifacts/phishing_detection_model.joblib`

The associated training summary is recorded in:

`artifacts/training_report.json`

4.2 Training Data and Preparation

The current model was trained using a PhiUSIIL-based prepared dataset. Training inputs include:

- `examples/phiusiil_model_features.csv`
- `examples/hard_negative_safe_samples.csv`

The training pipeline keeps the feature set compact so that model inference remains efficient for interactive use. The final selected artifact in the repository is an XGBoost model because it achieved the best balance of precision, recall, and ROC-AUC in the recorded evaluation. The presence of the training report in the repository also provides traceability for model selection rather than treating the active model as an undocumented implementation detail.

4.3 Feature Set

The final feature vector contains 12 features:

Feature	Purpose
URLLength	Captures unusually long URLs that may hide malicious intent
DomainLength	Measures domain length for structural irregularities
NoOfSpecialCharsInURL	Counts special characters often seen in deceptive URLs
IsDomainIP	Detects direct IP-based hosting instead of a named domain
NoOfSubDomain	Measures subdomain depth, which may indicate spoofing
HasSuspiciousPattern	Flags suspicious lexical patterns in the URL
SuspiciousPatternCount	Counts the number of suspicious URL patterns
UsesHTTPS	Indicates whether the URL uses HTTPS
NoOfPasswordFields	Captures password field presence in the webpage HTML
HasLoginForm	Detects login-form patterns in the webpage
HasExternalFormSubmit	Detects form submissions to external destinations
NoOfIFrames	Counts embedded iframe usage

The chosen feature set combines URL structure and page evidence. URL features capture lexical patterns such as length, special characters, and subdomain depth. Page features capture whether the website exposes signs such as login forms, password fields, external form submission, or iframe usage. This combination aligns the inference-time data shape with the prepared PhiUSIIL feature dataset used during training. This alignment is important because the project does not rely on lexical URL evidence alone; it also attempts to use observed webpage characteristics such as form structure and iframe usage whenever content retrieval succeeds.

4.4 Prediction Logic

The model service performs the following steps:

1. load the serialized model bundle
2. build a numeric feature frame using the expected feature order
3. compute phishing probability
4. derive an internal risk score
5. apply guardrails for weak webpage evidence and strong benign evidence
6. map the adjusted score to `Safe`, `Suspicious`, or `High Risk`

The backend stores the internal risk score for logging and admin reporting. The employee interface displays a derived safety score so the visual meaning is intuitive to users. This prevents the UI from exposing the internal risk semantics directly while keeping the database and admin records consistent with the model output. This adjustment was introduced to better match user expectations during interface testing, while preserving the internal model-facing semantics needed by the backend.

4.5 Model Thresholds

The current thresholds recorded in the training report are:

- suspicious threshold: `0.45`
- high-risk threshold: `0.90`

4.6 Recorded Test Metrics

The current test-set metrics for the selected XGBoost model are:

Metric	Value
Accuracy	0.9956992481203008
Precision	0.9945938818565401
Recall	0.9866579463701766
F1-score	0.9906100203559
ROC-AUC	0.999041840181793

The recorded metrics indicate that the selected model performs strongly on the prepared test split and is suitable for the current prototype release. The evaluation record also provides evidence that the chosen XGBoost artifact outperformed the alternative Logistic Regression and Random Forest baselines on the prepared dataset.

5 Backend Design

5.1 Application Entry Point

The backend application starts from `app/main.py` using Starlette. CORS is enabled for development so that the frontend can call the API directly during local testing. The route registration is centralized, which makes the integration behavior predictable and easy to maintain. The API layer is intentionally thin: request parsing, authorization checks, and error conversion happen at the route level, while the main business behavior is delegated to service and persistence components.

5.2 API Endpoints

5.2.1 Authentication

- `POST /api/auth/login`
- `GET /api/auth/session`
- `POST /api/auth/logout`

5.2.2 Employee Services

- `POST /api/analyze`
- `POST /api/reports`

5.2.3 Admin Services

- `GET /api/admin/overview`
- `GET /api/admin/cases`
- `GET /api/admin/scans`
- `GET /api/admin/models`

- `GET /api/admin/model/metrics`
- `GET /api/admin/feedback-dataset`
- `GET /api/admin/evaluations`

5.3 Error Handling

The backend returns structured JSON errors. Common failure cases include:

- invalid JSON request bodies
- invalid URL input
- missing or expired session tokens
- missing model artifact
- failed feature extraction
- internal processing errors

The API handlers convert these errors into explicit status codes so the frontend can present meaningful error states rather than failing silently. This improves system observability and makes regression testing easier because expected failure states are explicit.

6 Persistence Design

6.1 Database Layout

ClearURL stores data in separate SQLite files:

- `auth.sqlite3` for users and sessions
- `prediction_logs.sqlite3` for scan history
- `clearurl-feedback.sqlite3` for suspicious reports and cases
- `model_records.sqlite3` for model metadata and evaluation results

This separation improves maintainability by keeping authentication, prediction history, feedback, and model records logically isolated while still accessible to their corresponding services. It also reflects the project structure used by the different team contributions, which makes the handover simpler. It further reduces schema coupling, since operational logging, authentication, and model evaluation do not need to share a single evolving database schema.

6.2 Stored Data Semantics

- **Prediction logs**: request ID, normalized URL, risk score, label, model version, response time, timestamp
- **Feedback records**: report ID, case ID, normalized URL, report count, comments, report time
- **Model records**: algorithm name, version, dataset version, metrics, and evaluation metadata

The prediction log is important for both traceability and admin monitoring. The feedback store is important because it turns employee reports into reviewable cases that can later support future retraining or audit work. The model record store supports the administrative model summary and preserves benchmark information used during model comparison.

7 Frontend Design

7.1 User Experience Strategy

The frontend is designed around two distinct user roles:

- **Employee**: focused on URL analysis and reporting
- **Admin**: focused on operational monitoring and model visibility

The interface prioritizes clarity:

- green for safer results
- amber for suspicious results
- red for high-risk results
- explanation cards for risk transparency
- compact dashboard sections for admin monitoring

The employee page is intentionally focused on one task: analyse a URL and understand the result. The admin page is intentionally information-dense, because the main goal there is fast review of trends and cases rather than guided interaction. This separation of concerns supports role-based usability and keeps the primary employee task flow short.

7.2 Score Presentation Logic

To match user expectations, the employee-facing interface shows a safety-oriented score where higher values mean safer URLs. The backend still keeps the internal risk score for logging and analysis consistency. This avoids confusion for users while preserving the model-facing interpretation needed for logging and evaluation.

8 Testing and Validation

8.1 Automated Validation

The repository includes backend and frontend checks:

- `python -m pytest -q`
- `npm run build`

8.2 Recorded Evidence

The project also includes:

- `docs/TEST_RECORD.md`
- `docs/MODEL_TEST_RESULTS.md`
- `docs/ITERATION1_ACCEPTANCE_CRITERIA.md`

The repository already contains a dedicated testing record and model evaluation record, which supports the requirement for traceable iteration-level evidence. The automated test coverage in the repository includes authentication behavior, preprocessing validation, pipeline integration, training-related behavior, explainability integration, and admin API exposure.

9 Deployment and Runtime

9.1 Local Runtime

- frontend: Node.js 22+
- backend: Python 3.8+
- development server: `uvicorn app.main:app --host 127.0.0.1 --port 8000`
- frontend dev server: `npm run dev`

9.2 Operational Notes

- Some websites may block automated HTML retrieval.
- The system performs best when page content is available.
- The model artifact and SQLite files must remain available for the prototype to function correctly.

Because the project uses local SQLite files and a local serialized model bundle, a complete deployment or demo environment must include both the code and these runtime assets. For this reason, a deployable environment is not defined by application code alone; it also includes the model artifact and project data files.

10 Limitations and Future Improvements

Current limitations:

- webpage fetching can fail on sites that block automated requests
- suspicious cases are driven by employee reports rather than every scan
- the current admin console is read-only for operational review
- the safety score is a presentation-layer interpretation, not a separate model output

Potential future improvements:

- richer admin case workflows
- user scan history pages
- retraining support with corrected labels
- multilingual explanations
- campaign-level threat clustering

These items are appropriate for a later iteration because they extend the current prototype without changing the core employee analysis flow. They are also consistent with the future backlog direction already identified for the next iteration of the project.

11 Conclusion

ClearURL currently provides a working Iteration 1 prototype with authentication, analysis, explainability, reporting, admin monitoring, and a trained phishing detection model. The system is deployable as a full-stack application and has supporting test records, model evaluation records, and user-story documentation.